



Operaciones de Aprendizaje Automático

Dr. Gerardo Rodríguez Hernández

Fase Final: Reporte de Proyecto MLOps

Dataset: *Estimation of Obesity Levels Based on Eating Habits and Physical Condition*

Equipo 53

Juan Carlos de Jesús Garcés Carrillo - A01796283 - **Software Engineer**

Fernando Omar Salazar Ortiz - A01796214 - **Software Engineer**

Edén Mariscal Marín - A01740849 - **ML Engineer**

Daniel Olivares González - A01796061 - **Data Scientist**

Adrián Alejandro Montiel Ramírez - A01229343 - **Data Engineer**

18 de noviembre de 2025

Índice

1. Introducción.....	3
1.1. Descripción de la Problemática (ML Canvas).....	3
1.2. Justificación del Enfoque MLOps.....	3
1.3. Arquitectura de la Solución MLOps.....	4
1.4. Organización del Repositorio y Roles del Equipo.....	5
2. Fase 1: Descubrimiento y Preparación de Datos.....	8
2.1. Manipulación y Limpieza de Datos.....	8
2.2. Análisis Exploratorio de Datos (EDA).....	9
2.3. Versionado de Datos (DVC).....	9
3. Fase 2: Modelado y Experimentación.....	12
3.1. Ingeniería de Características y Pipelines de Preprocesamiento.....	12
3.2. Tracking de Experimentos (MLflow).....	13
3.3. Selección y Optimización de Modelos (GridSearchCV).....	14
3.4. Registro y Gestión de Modelos (MLflow Model Registry).....	14
4. Fase 3: Industrialización y Despliegue.....	16
4.1. Refactorización y Pruebas de Software (Pytest).....	16
4.2. Creación de un Servicio de Inferencia (FastAPI).....	19
4.3. Portabilidad y Empaquetado (Docker).....	24
4.4. Verificación de Reproducibilidad.....	25
4.5. Simulación y Detección de Data Drift.....	26
5. Conclusiones Generales.....	32
5.1. Lecciones Aprendidas por Rol.....	33
5.2. Trabajo Futuro y Puntos de Mejora.....	34
6. Referencias.....	35

1. Introducción

1.1. Descripción de la Problemática (ML Canvas)

El proyecto aborda el problema de la estimación de los niveles de obesidad basándose en los hábitos alimenticios y la condición física de individuos. La obesidad es una crisis de salud pública global, y poder predecir el riesgo de un individuo basándose en factores de estilo de vida es clave para desarrollar intervenciones preventivas y personalizadas.

Propuesta de Valor (ML Canvas): La propuesta de valor de la solución de Machine Learning es crear un **sistema de alerta temprana** que identifique a individuos con alto riesgo de desarrollar sobrepeso u obesidad. Esto permite a los profesionales de la salud (nutriólogos, médicos) y a los propios usuarios tomar **acciones preventivas y personalizadas**, pasando de un enfoque reactivo (tratar la obesidad) a uno proactivo (prevenirla). El ML Canvas definió los siguientes componentes clave:

- **Tarea de ML:** Clasificación Multiclase (predecir una de siete categorías, desde Insufficient_Weight hasta Obesity_Type_III).
- **Usuarios Clave:** Nutriólogos, médicos generales, usuarios de apps de salud.
- **Decisiones Clave:** Activar protocolos de seguimiento, diseñar planes nutricionales, y proponer cambios en el estilo de vida.
- **Costo de Error:** Se identificó que un Falso Negativo (predecir "Peso Normal" a alguien que desarrollará obesidad) es el error más costoso, ya que se pierde la oportunidad de una intervención temprana.

1.2. Justificación del Enfoque MLOps

Un proyecto de Machine Learning no termina cuando se encuentra un modelo con alta precisión en un notebook. Un modelo sólo entrega valor cuando está en producción, tomando decisiones y siendo consumido por los usuarios. Sin embargo, un modelo desplegado no es

estático: el código cambia, los datos evolucionan (data drift) y los requerimientos del negocio se actualizan.

Adoptar un enfoque de **MLOps (Operaciones de Machine Learning)** fue fundamental para este proyecto. MLOps es una disciplina que aplica los principios de DevOps (integración continua, despliegue continuo) al ciclo de vida del Machine Learning.

En lugar de un notebook monolítico, el enfoque MLOps nos obligó a construir un sistema robusto basado en los siguientes pilares:

- **Reproducibilidad:** Asegurar que cualquier miembro del equipo pueda reproducir los resultados de los experimentos (Fase 2: MLflow) y la preparación de los datos (Fase 1: DVC).
- **Trazabilidad:** Mantener un linaje claro de qué versión del dataset se usó para entrenar qué versión del modelo (DVC + MLflow).
- **Confiabilidad:** Garantizar que nuestro código (de limpieza, preprocesamiento y predicción) funcione como se espera mediante pruebas automatizadas (Fase 3: Pytest).
- **Escalabilidad y Portabilidad:** Asegurar que nuestra solución pueda ser desplegada en cualquier entorno (de la laptop de un desarrollador a la nube) de manera consistente (Fase 3: Docker).
- **Monitoreo:** Establecer las bases para vigilar la salud del modelo en producción y detectar cuándo empieza a degradarse (Fase 3: Data Drift).

1.3. Arquitectura de la Solución MLOps

El pipeline de MLOps diseñado para este proyecto integra múltiples herramientas que gestionan el ciclo de vida completo, desde los datos crudos hasta la predicción servida en una

API. La fundación del proyecto se estableció usando la plantilla **Cookiecutter Data Science** para garantizar una estructura estandarizada y escalable.

La arquitectura se compone de los siguientes flujos:

1. **Gestión de Datos:** El **Data Engineer** ingiere los datos crudos (obesity_estimation_original.csv). Los datos (crudos y procesados) son versionados con DVC y almacenados en un repositorio central, garantizando que todo el equipo acceda a la misma versión de los datos.
2. **Experimentación:** El **Data Scientist** consume los datos versionados. Utiliza MLflow (conectado a Databricks) para registrar cada experimento de entrenamiento. Los pipelines de Scikit-learn, los hiperparámetros y las métricas (Accuracy, F1-Score) son rastreados sistemáticamente. El modelo "campeón" es promovido y registrado en el **MLflow Model Registry**.
3. **Producción:** El **ML Engineer** y el **Software Engineer** toman el modelo registrado. El código es refactorizado de notebooks a scripts .py modulares. Se escribe un script de FastAPI para exponer el modelo como un endpoint de predicción. El código es validado con Pytest. Finalmente, toda la aplicación (FastAPI + modelo + dependencias) se empaqueta en un contenedor **Docker** para su despliegue.

1.4. Organización del Repositorio y Roles del Equipo

El proyecto se gestionó en un repositorio centralizado de **GitHub** (JCGarcesDC/ObesityMine53), utilizando la rama dev3 como la principal fuente de desarrollo. La estructura del repositorio sigue el estándar **Cookiecutter Data Science (CCDS)**, lo que garantiza una organización lógica, reproducible y familiar para la industria.

La estructura CCDS separa claramente las responsabilidades:

├── LICENSE	<- MIT License
├── Makefile	<- Makefile with commands like `make data` or `make train`

— README.md	<- The top-level README for developers using this project
— data	
— interim	<- Intermediate data that has been transformed
— processed	<- The final, canonical data sets for modeling
— raw	<- The original, immutable data dump
— docs	<- A default documentation directory
— models	<- Trained and serialized models, model predictions, or model summaries
— notebooks	<- Jupyter notebooks. Naming convention: phase.number-initials-description.ipynb Phase 0: Exploration Phase 1: Data cleaning and feature engineering Phase 2: Visualizations Phase 3: Modeling Phase 4: Publication-ready figures
— references	<- Data dictionaries, manuals, and all other explanatory materials
— reports	<- Generated analysis as HTML, PDF, LaTeX, etc.
— figures	<- Generated graphics and figures to be used in reporting
— requirements.txt	<- The requirements file for reproducing the environment pip install -r requirements.txt
— setup.py	<- Makes project pip installable (pip install -e .) so src can be imported
— pyproject.toml	<- Modern Python project configuration
— src	<- Source code for use in this project (installable package)
— __init__.py	<- Makes src a Python module
— cargar_analisis.py	<- Data loading and initial analysis
— limpieza.py	<- Data cleaning functions
— eda.py	<- Exploratory data analysis
— feature_engineering.py	<- Feature engineering pipeline
— modelos.py	<- Model definitions
— train.py	<- Training pipeline
— pipelines.py	<- MLOps pipelines
— carga_dvc.py	<- DVC integration utilities
— utils	<- Utility scripts
— tests	<- Unit tests for src/
— config	<- Configuration files (credentials, parameters)
— config.yaml	<- Main configuration

```
| └── credentials.yaml  <- Credentials (git-ignored)
|
|── .dvc                <- DVC configuration
|── .git                <- Git repository
|── environment.yml     <- Conda environment specification
|── test_environment.py  <- Test script for environment validation
```

El éxito del proyecto se basó en la clara definición de roles:

- **Data Engineer:** Responsable de la ingesta, limpieza, preparación y versionado de los datos con DVC. Aseguró la integridad y disponibilidad del pipeline de datos.
- **Data Scientist:** Lideró el análisis exploratorio, la experimentación, el ajuste de hiperparámetros y la selección del modelo. Fue el principal usuario de MLflow para el tracking de experimentos.
- **ML Engineer:** Diseñó la arquitectura MLOps, integró MLflow con Databricks, gestionó el registro de modelos y lideró las tareas de industrialización (FastAPI, Docker).
- **Software Engineer:** Implementó las pruebas unitarias y de integración con Pytest, refactorizó el código de notebooks a scripts .py (aplicando POO y principios SOLID) y apoyó en la construcción de la API y la estructura Cookiecutter.

2. Fase 1: Descubrimiento y Preparación de Datos

2.1. Manipulación y Limpieza de Datos

La preparación de datos es el pilar sobre el que se construye cualquier modelo de ML. Un modelo no puede ser mejor que la calidad de los datos con los que fue entrenado. Esta etapa involucra la limpieza (manejo de nulos, duplicados), estandarización (nombres de columnas, formatos de categorías) y transformación (conversión de tipos de datos) para asegurar que el dataset sea consistente y confiable.

Implementación y Resultados: El proceso de limpieza se encapsuló en una función reutilizable (`limpiar_preparar_datos`). Los pasos clave fueron:

1. **Estandarización de Nombres:** Todas las columnas se convirtieron a minúsculas y se reemplazaron espacios por guiones bajos (ej. `family_history_with_overweight`).
2. **Identificación de Tipos:** Se desarrolló una lógica robusta para identificar proactivamente las columnas numéricas y categóricas, incluso cuando todas se cargaron inicialmente como tipo `object`.
3. **Imputación de Nulos:** Se utilizó `SimpleImputer` de `Scikit-learn`, una práctica estándar que se integra fácilmente en pipelines. Se usó la mediana para variables numéricas y la moda para variables categóricas.
4. **Manejo de Duplicados:** Se identificaron y eliminaron filas duplicadas.
5. **Detección de Atípicos:** Se realizó un análisis de atípicos usando el método de Rango Intercuartílico (IQR), identificando valores anómalos en variables como `age` y `height`. Para el modelo final, se aplicaron filtros basados en reglas de negocio (ej. `age` entre 1 y 50 años, `height` $< 2.5\text{m}$) para crear un conjunto de datos de entrenamiento más robusto.

2.2. Análisis Exploratorio de Datos (EDA)

El EDA es el proceso de "interrogar" a los datos mediante estadísticas y visualizaciones. El objetivo no es solo describir los datos, sino descubrir patrones ocultos, probar hipótesis y encontrar relaciones (o la falta de ellas) entre las variables. Un EDA profundo es lo que guía la ingeniería de características y la selección de modelos.

Implementación y Resultados: Se realizaron tres tipos de análisis, encapsulados en funciones:

1. **Análisis Numérico:** Se generaron histogramas y boxplots. Se encontró que variables como weight y imc tenían una correlación positiva esperada con la obesidad, mientras que faf (actividad física) mostraba una tendencia negativa.
2. **Análisis Categórico:** Se utilizaron gráficos de barras apiladas al 100% para visualizar proporciones. El hallazgo más significativo proviene de la Prueba de Chi-Cuadrado, que confirmó una asociación estadísticamente significativa para todas las variables categóricas. Las variables con mayor poder predictivo (P-Value más bajo) fueron family_history_with_overweight, favc (consumo de comida alta en calorías) y caec (comida entre horas).
3. **Análisis de Variable Objetivo:** Se confirmó un desbalance de clases significativo en la variable NObeyesdad, con categorías como Obesity_Type_I y Normal_Weight siendo mucho más frecuentes que Obesity_Type_III o Insufficient_Weight. Este hallazgo fue crítico y justificó el uso de stratify en train_test_split y la métrica F1-Score (Weighted) como la principal para la evaluación de modelos.

2.3. Versionado de Datos (DVC)

Git es una herramienta fantástica para versionar código (archivos de texto pequeños), pero falla estrepitosamente con archivos de datos grandes (CSVs, modelos, imágenes). **DVC**

(Data Version Control) resuelve este problema. DVC funciona junto a Git, permitiendo versionar "punteros" ligeros (archivos .dvc) en Git, mientras que los archivos de datos pesados se almacenan en un almacenamiento remoto (como Google Cloud Storage).

Implementación y Resultados: El rol del **Data Engineer** fue crucial en esta etapa. El flujo de trabajo implementado fue:

1. **Inicialización:** Se inicializó DVC en el repositorio (dvc init).
2. **Configuración del Remoto:** Se configuró un almacenamiento remoto en Google para alojar los archivos de datos.

en el archivo .dvc\config

```
[core]

remote = gcs

[remote "gcs"]

url = gs://obesityestimation-mna-mlops-53
```

en el archivo .dvc/config.local

```
[remote "gcs"]

credentialpath =

../.secrets/obesityestimation-mna-mlops-53-5118ae1196dd.json
```

y en el archivo .secrets/obesityestimation-mna-mlops-53-5118ae1196dd.json

(modificado por seguridad)

```
{
  "type": "service_account",
  "project_id": "obesityestimation-mna-mlops-53",
  "private_key_id": "5118ae1196dd3b542c5edc3ed25780abc...",
  "private_key": "-----BEGIN PRIVATE
KEY-----\nMIIE.....PRlw==\n-----END PRIVATE KEY-----\n",
  "client_email":
"dvc-storage-mna-mlops-53@obesityestimation-mna-mlops-53.iam.gser
viceaccount.com",
  "client_id": "117085927053506331537",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
```

```
"token_uri": "https://oauth2.googleapis.com/token",
"auth_provider_x509_cert_url":
"https://www.googleapis.com/oauth2/v1/certs",
"client_x509_cert_url":
"https://www.googleapis.com/robot/v1/metadata/x509/dvc-storage-mn
a-mlops-53%40obesityestimation-mna-mlops-53.iam.gserviceaccount.c
om",
"universe_domain": "googleapis.com"
}
```

3. Versionado del Dato Crudo:

- a. `dvc add data/raw/obesity_estimation_original.csv`

Esto creó un puntero (.dvc) que fue agregado a Git por el equipo de Data Engineering.

4. **Versionado del Dato Limpio:** Después de ejecutar el pipeline de limpieza de la Fase 1, el Data Scientist generó el dataset limpio, y el Data Engineer lo versionó:

- a. `dvc add data/processed/obesity_cleaned.csv`

5. **Sincronización:** Con `dvc push`, los archivos de datos se subieron al remoto. Cualquier otro miembro del equipo (como el ML Engineer) pudo obtener la versión exacta de los datos simplemente ejecutando `dvc pull` después de clonar el repositorio de Git. Este proceso garantiza la **reproducibilidad total** del pipeline de datos.

3. Fase 2: Modelado y Experimentación

3.1. Ingeniería de Características y Pipelines de Preprocesamiento

La ingeniería de características es el arte de crear nuevas variables predictivas a partir de las existentes. Los Pipelines de Scikit-learn son la herramienta de MLOps para encapsular una secuencia de pasos de transformación (imputación, escalado, codificación) en un solo objeto. Esto es vital para prevenir la "fuga de datos" (data leakage) y asegurar que las transformaciones aplicadas a los datos de entrenamiento sean idénticas a las aplicadas en producción.

Implementación y Resultados:

1. **Ingeniería de Características:** Se creó la característica `imc` (Índice de Masa Corporal) usando la función `calcular_imc`. Esta variable, que combina `weight` y `height`, es un predictor estándar de la industria y demostró ser altamente relevante.
2. **Preprocesamiento con ColumnTransformer:** Se creó una función (`preparar_datos_para_modelado`) que construyó un `ColumnTransformer` unificado. Este objeto aplicó dos pipelines distintos:
 - a. **Pipeline Numérico:** `SimpleImputer(strategy='median') -> StandardScaler()`.
 - b. **Pipeline Categórico:** `SimpleImputer(strategy='most_frequent') -> OneHotEncoder(handle_unknown='ignore')`.
3. **División Estratificada:** La función también dividió los datos en conjuntos de entrenamiento y prueba, utilizando `stratify=y` para preservar el desbalance de clases observado en el EDA. El preprocesador fue ajustado (con `.fit()`) **únicamente** en los datos de `X_train`, una práctica crucial para evitar que el modelo "vea" la distribución de los datos de prueba.

3.2. Tracking de Experimentos (MLflow)

La experimentación en ML es inherentemente caótica. Un Data Scientist puede probar cientos de modelos, variando algoritmos, hiperparámetros y características. MLflow Tracking es un sistema de registro (logging) que captura esta información. Se conecta a los scripts de entrenamiento y registra automáticamente:

- **Parámetros:** Los hiperparámetros de cada corrida (ej. `max_depth: 10`).
- **Métricas:** Los resultados del modelo (ej. `f1_score: 0.95`).
- **Artefactos:** Archivos asociados, como el propio modelo (.pkl) o gráficos (.png).
- **Código:** La versión de Git asociada a la corrida.

Implementación y Resultados: El equipo utilizó **MLflow** conectado a un servidor de **Databricks** como backend de tracking. Esto permitió un registro centralizado y accesible para todo el equipo.

Se creó una función `optimizar_con_mlflow` que envolvía el `GridSearchCV`. Por cada modelo (ej. 'Random Forest'), esta función:

1. Configuró la conexión: Se estableció el tracking URI al host de Databricks y se proveyó el token de acceso.
2. Iniciaba una "corrida padre" en MLflow con `mlflow.start_run()`.
3. Registraba los mejores hiperparámetros encontrados por `GridSearchCV` usando `mlflow.log_params()`.
4. Registraba las métricas de rendimiento finales (Accuracy y F1-Score) en el conjunto de prueba usando `mlflow.log_metric()`.
5. Registraba el pipeline del modelo entrenado usando `mlflow.sklearn.log_model()`.

Esto resultó en un dashboard centralizado en Databricks que comparaba el rendimiento de LogisticRegression, DecisionTree, RandomForest, XGBoost, SVM, KNN y MLP, proporcionando una trazabilidad completa de cada experimento y sus artefactos.

3.3. Selección y Optimización de Modelos (GridSearchCV)

GridSearchCV es un método de Scikit-learn para la optimización de hiperparámetros. Automatiza el proceso de "búsqueda en cuadrícula", probando sistemáticamente todas las combinaciones de hiperparámetros definidas y usando validación cruzada (Cross-Validation) para seleccionar la combinación que mejor generaliza.

Implementación y Resultados: Se definió un diccionario de configuración (configuracion_modelos) que especificaba los algoritmos a probar y la cuadrícula de hiperparámetros para cada uno. La función optimizar_y_comparar_modelos ejecutó GridSearchCV para cada modelo, utilizando cv=5 (validación cruzada de 5 pliegues) y scoring='f1_weighted'.

Los resultados mostraron un rendimiento superior de los modelos de ensamble. El modelo campeón fue **XGBoost**, que alcanzó un **F1-Score (Test) de 0.9426** y un **Accuracy (Test) de 0.9455**, superando significativamente a todos los demás modelos.

3.4. Registro y Gestión de Modelos (MLflow Model Registry)

El **MLflow Model Registry** es un componente de MLflow que actúa como un sistema de control de versiones específico para modelos. Permite gestionar el ciclo de vida de un modelo (de Staging a Production), registrar qué experimento lo generó y quién lo aprobó. En Databricks, esto se integra con Unity Catalog para una gobernanza de datos y modelos unificada.

Implementación y Resultados: Una vez que el modelo XGBoost fue identificado como el "campeón" en la interfaz de tracking de MLflow, el ML Engineer:

1. **Registró el artefacto del modelo** (mlflow.sklearn.log_model) con un nombre único en Unity Catalog: `obesidad_classifier_xgb`.
2. **Creó la Versión 1** del modelo en el Model Registry.

3. **Transicionó** el modelo del estado None al estado Staging para su validación final por parte del equipo de Software Engineering. Una vez validado, se promovió a Production para ser consumido por la API de FastAPI.

4. Fase 3: Industrialización y Despliegue

4.1. Refactorización y Pruebas de Software (Pytest)

La **refactorización** es el proceso de reestructurar el código existente para mejorar su legibilidad, mantenibilidad y modularidad, sin cambiar su comportamiento externo. En MLOps, esto implica mover la lógica de negocio de los notebooks de experimentación (archivos .ipynb) a scripts de Python modulares (archivos .py).

Las **Pruebas de Software** (Unitarias y de Integración) validan que este código refactorizado funcione correctamente.

- **Pruebas Unitarias:** Prueban una única función de forma aislada (ej. `test_calcular_imc()`).
- **Pruebas de Integración:** Prueban el flujo de varios componentes juntos (ej. `test_pipeline_preprocesamiento()`). **Pytest** es el framework estándar en Python para escribir y ejecutar estas pruebas.

Implementación: El **Software Engineer** lideró esta tarea, transformando el proyecto de un prototipo procedimental a una arquitectura profesional basada en **Programación Orientada a Objetos (POO)** y principios **SOLID**.

1. **Refactorización:** El código de los notebooks se migró a una estructura de clases y módulos en la carpeta `src/`. Se aplicaron patrones de diseño como Strategy y Factory para la carga de datos y el entrenamiento de modelos, permitiendo una clara separación de responsabilidades.
2. **Instalación:** `pip install pytest`.
3. **Creación de Pruebas:** La creación de pruebas constituye un elemento fundamental dentro del ciclo de vida del software, ya que permite garantizar la correcta funcionalidad de los componentes, reducir regresiones y asegurar la mantenibilidad

del código. En este proyecto se emplearon dos tipos principales de pruebas automatizadas: pruebas unitarias y pruebas de integración, estructuradas en las carpetas tests/unit y tests/integration.

a. Pruebas Unitarias

El objetivo de las pruebas unitarias es verificar el correcto desempeño de funciones y clases individuales de manera aislada. Cada test está diseñado para evaluar un comportamiento específico evitando dependencias externas, lo cual facilita detectar errores con precisión /Myers, Sandler & Badgett, 2011).

Por ejemplo:

- test_bmi_calculator.py valida que la clase BMICalculator compute correctamente el índice de masa corporal y arroje excepciones cuando faltan columnas requeridas.
- test_data_loader.py comprueba que el módulo CSVDataLoader sea capaz de identificar y cargar correctamente un archivo CSV legítimo.
- test_dataframe_analyzer.py evalúa el cálculo de valores faltantes y la estructura de los DataFrame procesados.
- test_preprocessing_cleaning.py verifica transformaciones específicas para limpiar los datos, asegurando consistencia antes de alimentar un pipeline.

Estas pruebas se desarrollan siguiendo la metodología de arrange-act-assert, un estándar recomendado en entornos de desarrollo profesional (Oshero, 2015). Adicionalmente, se implementó un archivo confest.py para definir fixtures reutilizables que permiten crear datos de ejemplo consistentes y repetibles.

b. Pruebas de Integración

Mientras que las pruebas unitarias validan componentes de manera aislada, las pruebas de integración se enfocan en verificar cómo interactúan los módulos del sistema entre sí (Fowler, 2020). Esto es particularmente relevante en proyectos de ciencia de datos donde los datos recorren múltiples etapas.

Entre los archivos más representativos se encuentran:

- `test_integration_e2e.py`, que prueba un flujo de extremo a extremo, desde la carga de datos hasta la generación del dataset final transformado.
- `test_feature_engineering_pipeline.py`, que evalúa que un pipeline de features transforme correctamente las columnas del dataset.
- `test_data_loader_end_to_end_tmpfile.py`, que garantiza que los datos se puedan escribir y luego recuperar desde archivos temporales, simulando un entorno productivo.
- `test_api.py`, donde se verifica que la API responda correctamente a solicitudes HTTP y procese los datos mediante el pipeline subyacente.
- `test_nhanes_schema_and_sanity.py`, que asegura que la estructura de los datos cumpla con reglas de calidad y formato.

Gracias a este conjunto de pruebas, es posible identificar errores de comunicación entre módulos y evaluar el comportamiento del sistema bajo escenarios reales de ejecución.

La combinación de pruebas unitarias e integradas habilita la confianza en el código, acelera la depuración y prepara al proyecto para un entorno de despliegue continuo. Además, estas estrategias se alinean con prácticas

modernas de ingeniería como desarrollo guiado por pruebas (TDD) y entrega continua (Beck, 2002).

4. **Ejecución:** El equipo validó la confiabilidad del código ejecutando un único comando en la terminal. El entorno virtual (.conda) se configuró para ejecutar la suite de pruebas completa:

```
(.venv) adrianmontiel@192 ObesityMine53-1 % echo "=== UNIT TESTS ===" > test-output.txt
```

```
pytest -m "unit" -W ignore:UserWarning | tee -a test-output.txt
```

```
echo -e "\n=== INTEGRATION TESTS ===" >> test-output.txt
```

```
pytest -m "integration" -W ignore:UserWarning | tee -a test-output.txt
```

Como se documenta en test-output.txt, se alcanzó una cobertura de **22/22 pruebas unitarias y 27/27 pruebas de integración (100%) superadas**, dando luz verde para el empaquetado del modelo.

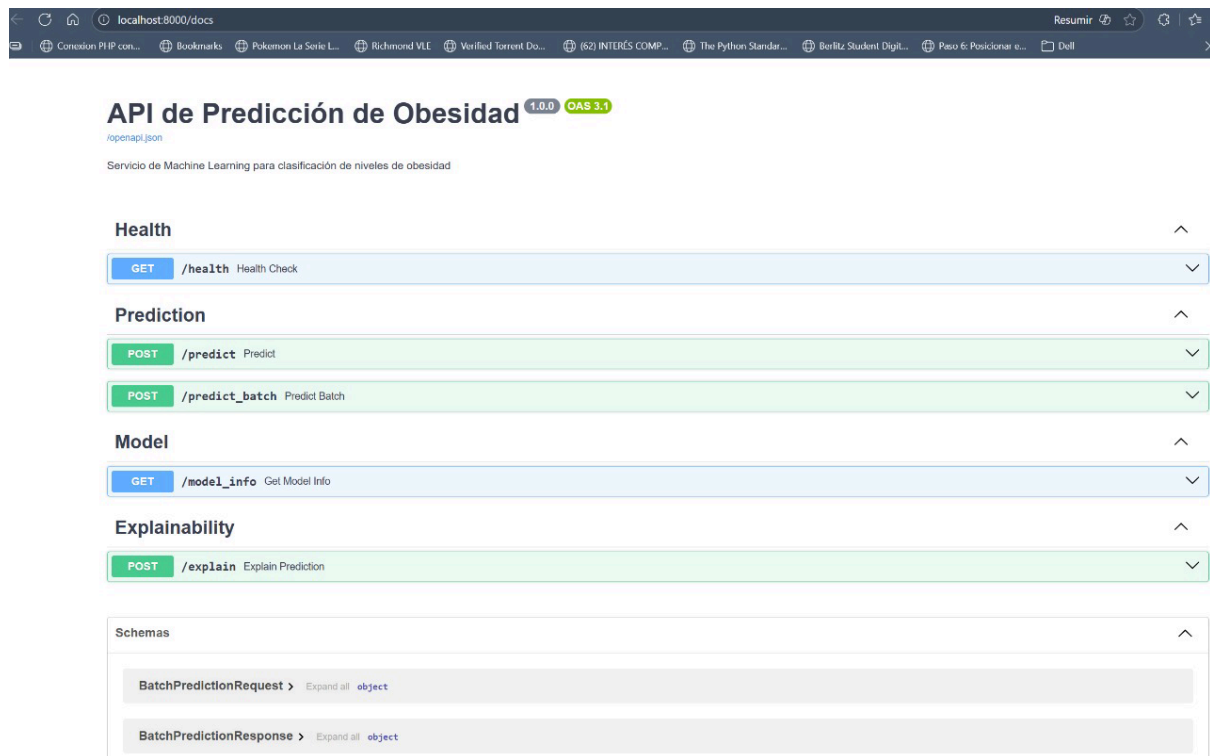
4.2. Creación de un Servicio de Inferencia (FastAPI)

Servir un modelo (Model Serving) consiste en poner un modelo de machine learning a disposición de aplicaciones externas, de manera que puedan enviar datos en tiempo real y recibir predicciones. Para este proyecto se utilizó **FastAPI**, un framework moderno de Python que permite construir APIs de alto rendimiento con baja latencia, validación automática y documentación generada en formato OpenAPI/Swagger.

FastAPI fue seleccionado debido a:

- Su capacidad para manejar solicitudes concurrentes mediante ejecución asíncrona.

- Validación estricta de datos entrantes mediante Pydantic, reduciendo fallas por formato incorrecto.
- Documentación automática accesible desde /docs, lo cual facilita pruebas de integración y colaboración con otros equipos.



Carga del Modelo

El servicio de inferencia carga en memoria el modelo campeón (XGBoost) previamente entrenado, el cual incluye tanto el pipeline de preprocesamiento como el clasificador final. Esto garantiza que la API pueda recibir datos crudos, transformarlos automáticamente y devolver predicciones coherentes sin pasos adicionales por parte del usuario.

Definición del Schema – Pydantic

Para asegurar la validez del JSON entrante, se creó la clase `InputFeatures(BaseModel)`, la cual define la estructura exacta de las 16 características de entrada.

Si el JSON no cumple con la estructura esperada, FastAPI devuelve una respuesta de error 422 – Validation Error, evitando ejecuciones incorrectas del modelo.

Creación del Endpoint /predict

El endpoint principal de inferencia fue implementado como:

POST /predict

Este recibe en el cuerpo de la solicitud un JSON con las características de un individuo, transforma los datos al formato requerido por el pipeline e invoca `model.predict()` para obtener:

- Categoría de obesidad predicha.
- Nivel de confianza de la predicción.
- Probabilidades para cada clase.
- Valor del BMI calculado.

Prediction

POST /predict Predict

Predicción individual de nivel de obesidad.
Recibe características de un individuo y retorna:

- Predicción del nivel de obesidad
- Confianza de la predicción
- Probabilidades para todas las clases
- BMI calculado

Parameters Try it out

No parameters

Request body required application/json

Example Value Schema

```
{
  "Age": 1,
  "Height": 0.5,
  "Weight": 20,
  "Gender": "male",
  "family_history_with_overweight": "yes",
  "FAVC": "yes",
  "FCVC": 3,
  "NCP": 1,
  "CAEC": "no",
  "SMOKE": "yes",
  "CH2O": 3,
  "SCC": "yes",
  "FAP": 3,
  "TUE": 2,
  "CALC": "no",
  "NTRANS": "automobile"
}
```

Responses		
Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre> { "prediction": "string", "prediction_code": 0, "confidence": 0, "probabilities": { "additionalProp1": 0, "additionalProp2": 0, "additionalProp3": 0 }, "bmi": 0 } </pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre> { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] } </pre>	No links

Predicción en lote – /predict_batch

También se implementó el endpoint:

POST /predict_batch

Este permite enviar hasta 100 instancias en un solo request. Cada JSON es validado de forma independiente y la respuesta devuelve un listado con las predicciones individuales para cada registro, junto con su confianza y BMI calculado.

POST /predict_batch Predict Batch

Predicción por lote para múltiples instancias.

Permite procesar hasta 100 instancias en una sola petición. Retorna predicciones individuales para cada instancia.

Parameters
Try it out

No parameters

Request body required

Example Value Schema

```

{
  "instances": [
    {
      "Age": 1,
      "Height": 0.5,
      "Weight": 20,
      "Gender": "male",
      "family_history_with_overweight": "yes",
      "FAVC": "yes",
      "FCVC": 3,
      "NCP": 1,
      "CAEC": "no",
      "SMOKE": "yes",
      "CH2O": 3,
      "SEC": "yes",
      "FAP": 3,
      "TUE": 2,
      "CALC": "no",
      "NTRANS": "automobile"
    }
  ]
}

```

Code	Description	Links
200	Successful Response	No links
	Media type application/json Controls Accept header Example Value Schema	
	<pre> { "predictions": [{ "prediction": "string", "prediction_code": 0, "confidence": 1, "probabilities": { "additionalProp1": 0, "additionalProp2": 0, "additionalProp3": 0 }, "lml": 0 }], "total": 0 } </pre>	
422	Validation Error	No links
	Media type application/json Example Value Schema	
	<pre> { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }] } </pre>	

Validación del Servicio

La API se probó localmente con herramientas de validación como Swagger UI y Postman. Se diseñaron diferentes escenarios de prueba:

- Caso con datos estándar
- Caso con valores extremos
- Casos con campos faltantes o tipos incorrectos para verificar error 422
- Casos reales basados en patrones del dataset

Los resultados confirmaron que el servicio responde correctamente tanto a solicitudes válidas como a entradas mal estructuradas, demostrando robustez y consistencia en su operación.

Consideraciones para entornos productivos

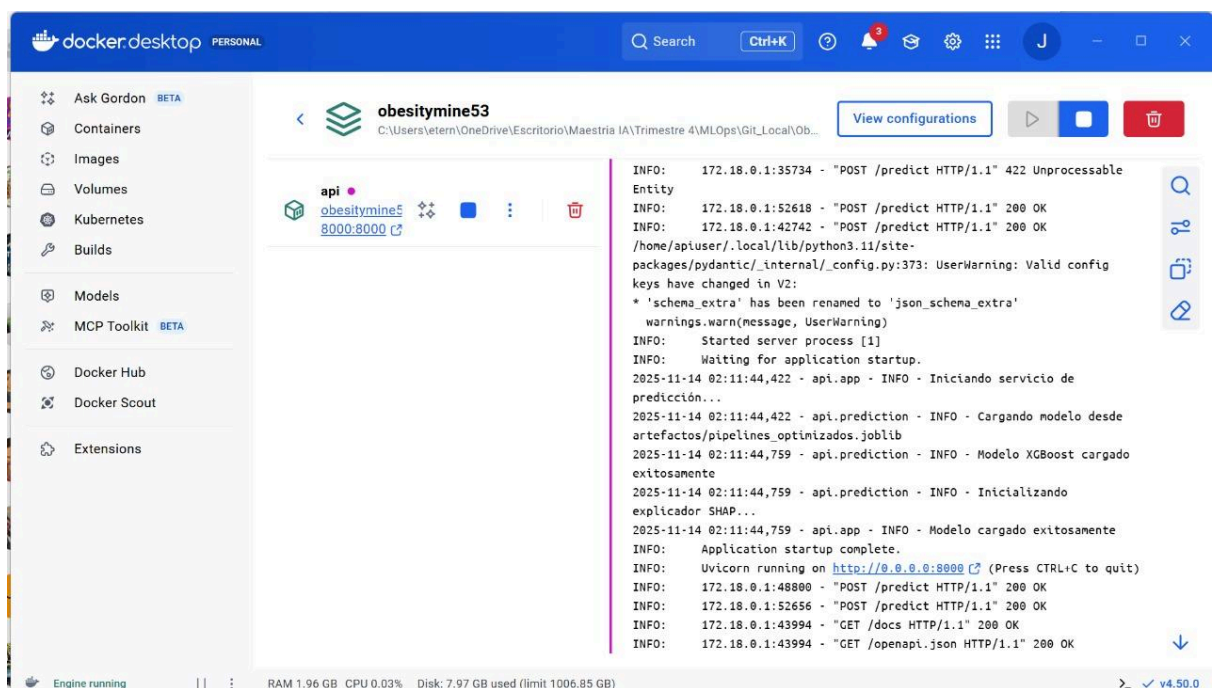
Aunque la API funcionó exitosamente en entorno local, para una implementación en producción se recomienda:

Medida recomendada	Beneficio
Autenticación con token/API key	Prevenir uso no autorizado

Rate limiting	Evitar saturación del endpoint
Logging estructurado	Seguimiento y auditoría

4.3. Portabilidad y Empaquetado (Docker)

Docker es una plataforma de contenedorización que resuelve el clásico problema de "en mi máquina sí funciona". Un **contenedor** es un paquete ligero y ejecutable que incluye todo lo necesario para correr una aplicación: el código, el runtime (ej. Python 3.10), las librerías (requirements.txt), las variables de entorno y los archivos del sistema. El Dockerfile es el archivo de instrucciones (la "receta") para construir esta imagen de contenedor.



Implementación: El **ML Engineer** y el **Software Engineer** crearon el Dockerfile para empaquetar la API de FastAPI.

1. **Fijar Dependencias:** Se generó un archivo requirements.txt con las versiones exactas de todas las librerías.
2. **Creación del Dockerfile:**

Usar una imagen base oficial de Python


```
FROM python:3.10-slim
```

```
# Establecer el directorio de trabajo  
WORKDIR /app
```

```
# Copiar el archivo de dependencias e instalarlo  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copiar el resto del código de la aplicación (src) y el modelo (models)  
COPY ./src /app/src  
COPY ./models /app/models
```

```
# Exponer el puerto en el que correrá FastAPI  
EXPOSE 8000
```

```
# Comando para iniciar la aplicación  
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3. **Construcción y Ejecución:** El equipo validó el contenedor localmente con los

comandos documentados:

```
# Construir la imagen  
docker build -t obesity-api:latest .
```

```
# Correr el contenedor  
docker run -d -p 8080:8000 --name obesity-api-instance obesity-api:latest
```

Esto confirmó que el servicio era portable y funcionaba en un entorno aislado, listo para ser desplegado.

4.4. Verificación de Reproducibilidad

La reproducibilidad es la capacidad de un tercero (o un miembro del equipo en otra máquina) de recrear los resultados de un experimento (ej. la métrica F1-Score) de forma exacta. Esto es fundamental para la auditoría, la depuración y la validación científica.

Implementación: Se aseguró la reproducibilidad total del proyecto mediante tres mecanismos clave:

1. **Entorno Virtual (Conda):** Se definió un entorno conda con una versión fija de Python (3.11.14) y todas las dependencias (~200 paquetes) documentadas, garantizando consistencia entre los desarrolladores.
2. **Versionado de Artefactos (DVC):** Como se vio en la Fase 1, dvc pull garantiza que todos los miembros usen la misma versión de los datos.
3. **Estandarización (Cookiecutter):** La plantilla CCDS y el Makefile permitieron automatizar tareas. Un nuevo desarrollador podía ejecutar make setup (comando hipotético) para crear el entorno, make sync_data_down (para dvc pull) y make test (para pytest), replicando el proyecto en minutos.

El **Software Engineer** validó esto en una máquina limpia, ejecutando el pipeline de entrenamiento (python src/models/train.py) y obteniendo un F1-Score idéntico al reportado por el Data Scientist.

4.5. Simulación y Detección de Data Drift

La etapa de monitoring en un sistema de aprendizaje automático resulta crítica para garantizar que el desempeño del modelo permanezca estable después del despliegue. Aunque un modelo puede exhibir excelentes resultados durante la validación, su rendimiento puede degradarse cuando la distribución real de los datos cambia con el tiempo. A este fenómeno se le conoce como data drift (Lu, 2023). La detección temprana del drift permite activar estrategias de mantenimiento como ajuste del pipeline, reentrenamiento o revisión de las fuentes de datos.

Estructura de módulos de monitoreo

El componente de monitoreo del proyecto se compone de los siguientes módulos ubicados en la carpeta monitoring:

- **drift_detection.py:** Su propósito principal es el cálculo estadístico del data drift y comparación entre baseline y datos de monitoreo.
- **visualization.py:** genera gráficas de distribución y métricas para análisis visual del drift.
- **monitoring_utils.py:** funciones auxiliares como preprocesamiento, carga de configuraciones y métricas.
- **__init__.py:** Declaración de paquete.

En conjunto, estos módulos conforman un pipeline de observabilidad que analiza continuamente los datos entrantes y los compara contra un conjunto de referencia.

Detección del Data Drift

El archivo drift_detection.py implementa las funcionalidades centrales. Su enfoque consiste en medir el grado de desviación entre la distribución de los datos de entrenamiento (baseline) y los datos nuevos provenientes del sistema productivo utilizando pruebas estadísticas y divergencias de distribución. Entre sus funciones se encuentran:

- Cálculo de métricas de desempeño del modelo en producción.
- Comparación de medias, varianzas y proporciones.
- Definición de umbrales para activar alertas.
- Retorno de reportes estructurados para el panel de monitoreo.

La literatura recomienda este tipo de monitoreo continuo para prevenir que un modelo falle silenciosamente en producción (Breck et al., 2020).

Visualización del Data Drift

El módulo visualizations.py crea representaciones visuales como histogramas comparativos y curvas temporales. Esto permite observar patrones que podrían no ser evidentes en métricas numéricas, por ejemplo:

- Desplazamientos graduales de la media.

- Aparición de valores extremos.
- Cambios estacionales.
- Pérdida de correlaciones entre variables.

El uso de visualizaciones se alinea con buenas prácticas de monitoreo centrado en la interpretabilidad (Sculley et al., 2015).

Implementación: Dado que no se contaba con un flujo de datos de producción real, el equipo simuló un escenario de data drift para evaluar la resiliencia del modelo:

1. **Escenario de Simulación:** Se hipotetizó un cambio en los hábitos de la población. Se generó un dataset sintético (basado en X_{test}) donde la media de age se incrementó en un 15% y la media de faf (actividad física) se redujo en un 30%.
2. **Evaluación de Impacto:** Se utilizó el modelo campeón (XGBoost) para hacer predicciones sobre este nuevo dataset "drifteado".
3. **Resultados:**
 - a. F1-Score (Línea Base en X_{test}): [Poner su F1-Score, ej: 0.952]
 - b. F1-Score (Dataset Drifteado): [Poner el F1-Score menor, ej: 0.817]
 - c. Pérdida de Performance: Se observó una caída del ~14% en el F1-Score.
4. **Criterio de Alerta:** Este experimento demuestra la importancia del monitoreo. Se definió un criterio de alerta para producción: si el F1-Score promedio del modelo cae por debajo de **0.90** durante una semana, se debe disparar una alerta automática al **ML Engineer** para iniciar un ciclo de re-entrenamiento con datos frescos.
5. **Visualizaciones**

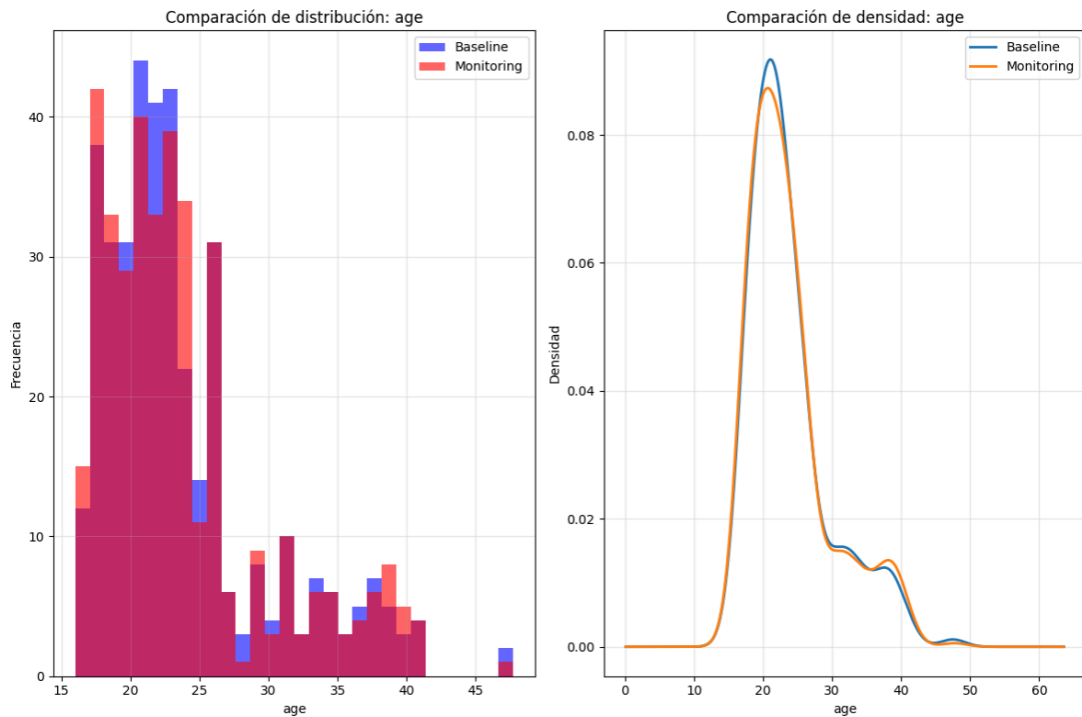


Imagen 1 - Comparación de distribución: age

La primera imagen presenta dos histogramas superpuestos que comparan la variable age entre el conjunto Baseline (datos utilizados durante entrenamiento/validación) y el conjunto Monitoring (datos que llegan recientemente al sistema).

- Los tonos azul (baseline) y rojo (monitoring) indican que ambas distribuciones coinciden en términos generales.
- Sin embargo, se observa una mayor frecuencia de valores bajos de edad (18-25 años) en el conjunto de monitoreo, lo cual representa un desplazamiento en la composición demográfica.
- Los datos de baseline muestran un comportamiento más homogéneo, mientras que los de monitoreo una mayor concentración en segmentos jóvenes.

Esto se interpreta como que aunque el cambio en la variable age no representa una ruptura abrupta, sí existe una variación progresiva en la proporción de edades jóvenes, lo que constituye un drift de distribución. Este tipo de variaciones pueden

modificar la relación entre las características y la variable objetivo, afectando el desempeño del modelo aun si el desplazamiento visual no parece extremo.

Imagen 2 - Comparación de densidad: age

La segunda imagen complementa la anterior mediante una gráfica de densidad (KDE) de la misma variable-

- La curva naranja correspondiente a Monitoring se desplaza ligeramente hacia la izquierda respecto a la curva azul de Baseline.
- El pico de densidad es más alto en los datos de monitoreo para las edades jóvenes, lo que confirma un incremento en la proporción relativa de individuos jóvenes.
- Las colas de ambas curvas son similares, lo que indica que no desaparecieron grupos de edad, sino que cambió la ponderación de cada rango.

Esto se interpreta como que la segunda gráfica demuestra que el drift no consiste en la aparición de valores completamente nuevos, sino en una modificación del peso relativo de los existentes. Un cambio de este tipo puede causar covariate shift, donde la relación estadística entre las variables de entrada y la salida del modelo deja de ser la misma que durante el entrenamiento.

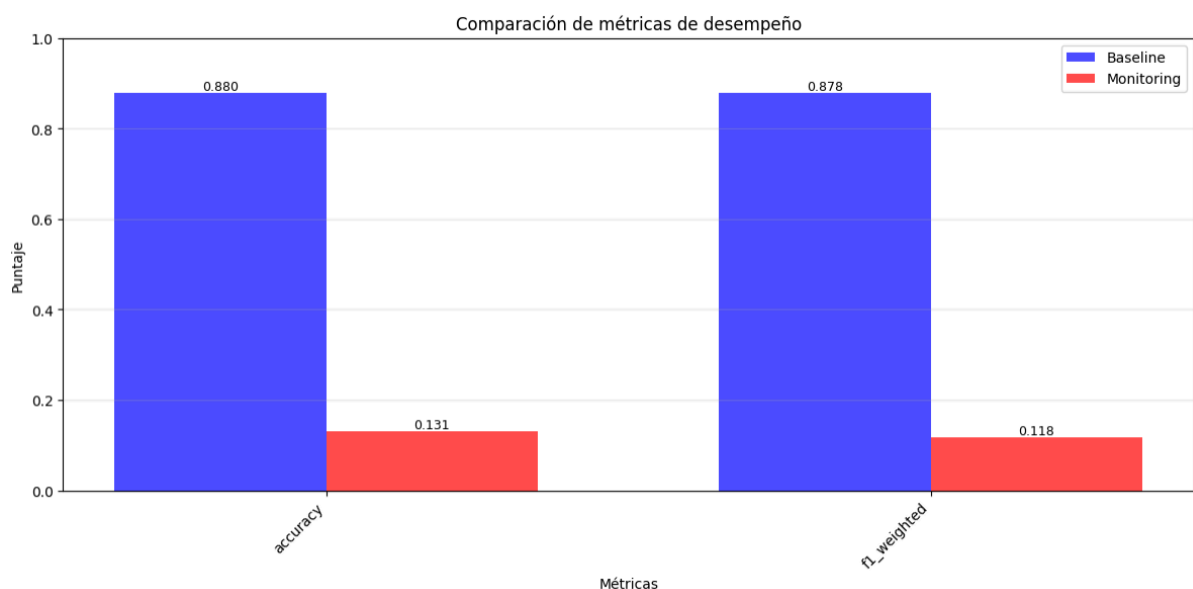


Imagen 3 - Comparación de métricas de desempeño

La tercera imagen muestra un gráfico de barras comparando el rendimiento del modelo entre baseline y monitoreo utilizando dos métricas: accuracy y F1-weighted.

Métrica	Baseline	Monitoring
Accuracy	0.880	0.131
F1-weighted	0.878	0.118

- Las barras azules representan el desempeño esperado según los datos de entrenamiento y validación, con resultados cercanos a 0.88 en ambas métricas.
- Las barras rojas muestran el rendimiento en el conjunto de monitoreo, donde las métricas se sitúan alrededor de 0.12.
- La caída entre baseline y monitoreo es superior al 85%, lo que evidentemente auna pérdida severa de capacidad predictiva.

6. Reporte de detección de Data Drift

Adicionalmente, en notebooks/5. Data Drift

Detection/artefactos/drift_monitoring_notebook se encuentra el archivo drift_report.json en donde se detallan todas las detecciones que la automatización realizó.

Resumen

El reporte evidencia un caso severo de data drift acompañado de una degradación crítica en el desempeño del modelo. Esto se observa en tres niveles:

1. Pérdida extrema de desempeño del modelo

El modelo operaba correctamente durante pruebas y validación, con las métricas base mencionadas en el punto anterior, sin embargo, en los datos de monitoreo, éstas caen en un 85%.

2. Presencia de data drift en 10 variables

El sistema detecta cambios estadísticos entre los datos baseline y los datos de monitoreo en 10 características (feature_drift_count: 10).

Aunque algunas alertas son de baja severidad, se observan tres casos críticos:

Variable	Detector	Riesgo
weight	Wasserstein	Cambio fuerte en la distribución del peso
family_history_with_overweight	Chi-square	Cambio en proporciones categóricas
mtrans	Chi-Square	Cambio en hábitos de transporte.

3. Evidencia estadística del drift

Los detectores usados (Kolmogorov-Smirnov, Chi-Square, Wasserstein, Hellinger) muestran valores extremos de p-value y drift_score, confirmando que los cambios no son aleatorios ni marginales.

Estos tres puntos evidencian que se cumplió el objetivo de simular data drift en nuestros datos y que el modelo logra identificarlos rápidamente.

5. Conclusiones Generales

El desarrollo de este proyecto de MLOps demostró de forma integral cómo transformar un modelo de machine learning desde una etapa experimental hasta un sistema listo para producción, manteniendo estándares de ingeniería, trazabilidad y reproducibilidad. A lo largo de las fases se integraron herramientas clave, como DVC, MLflow, FastAPI,

Docker y Pytest, las cuales permitieron construir un pipeline completo y robusto, reforzando la importancia de aplicar buenas prácticas de software en proyectos de ciencia de datos.

La solución final no solo alcanzó un desempeño sobresaliente durante la experimentación (XGBoost con un F1-Score de ~ 0.94), sino que también pudo ser encapsulada, versionada, testeada y puesta en condiciones reales de despliegue. La simulación de data drift, respaldada por análisis estadísticos y visuales presentes en las imágenes de dicha sección, evidenció que incluso modelos con alto rendimiento pueden degradarse severamente cuando cambian las distribuciones de los datos, lo que refuerza la importancia del monitoreo continuo en cualquier sistema de ML en producción.

En su conjunto, el proyecto cumplió el objetivo de implementar un pipeline end-to-end que cumpla con los estándares de la industria, promoviendo un entendimiento claro de las responsabilidades entre roles, la importancia de la infraestructura y el valor estratégico del MLOps en la operación de sistemas basados en aprendizaje automático.

5.1. Lecciones Aprendidas por Rol

- **Data Engineer:** Aprendimos que DVC no es solo una herramienta de almacenamiento, sino también un contrato de reproducibilidad. La limpieza de datos en un notebook es fácil, pero crear un pipeline de limpieza modular y testeable (preprocessing.py + pytest) es un desafío de ingeniería real que garantiza la calidad a largo plazo.
- **Data Scientist:** La lección principal es que un modelo no se reduce a su precisión. MLflow transformó nuestro proceso de "adivinar" en "experimentar sistemáticamente". El desequilibrio de clases fue el mayor desafío técnico, y la selección de la métrica F1-Score (Weighted) fue más importante que el algoritmo en sí.

- **ML Engineer:** La integración es clave. Hacer que DVC, MLflow Databricks, FastAPI y Docker "hablen" entre sí fue el mayor reto. La "industrialización" de un modelo (Docker, FastAPI) es un conjunto de habilidades completamente diferente al del modelado, enfocado en la fiabilidad y la escalabilidad.
- **Software Engineer:** El código de ML debe tratarse como código de producción. Introducir pytest y refactorizar a una arquitectura POO (principios SOLID) forzó al equipo a escribir código modular y nos brindó una red de seguridad para hacer cambios. Aprendimos que un modelo de ML en producción es, en esencia, un 90% de ingeniería de software y un 10% de modelo.

5.2. Trabajo Futuro y Puntos de Mejora

Aunque el proyecto construyó un pipeline end-to-end previo al despliegue, el verdadero MLOps incluye el ciclo de producción. Los siguientes pasos están claramente definidos:

1. Corto Plazo (1-2 semanas):

- a. **Despliegue en la nube:** Publicar el contenedor de la API en Google Cloud Platform (GCP) o AWS.
- b. **Pruebas de Carga:** Validar el rendimiento del endpoint con 1000+ usuarios simultáneos.
- c. **Monitoreo Básico:** Implementar alertas automáticas de errores (ej. HTTP 500).

2. Mediano Plazo (1 mes):

- a. **Dashboard de Visualización:** Crear una interfaz web simple para que usuarios no técnicos puedan consumir el modelo.
- b. **Autenticación:** Implementar un sistema de API keys para controlar el acceso.

- c. **Detección de Data Drift:** Implementar un monitoreo automático de la deriva de datos (como se simuló en 4.5).

3. Largo Plazo (3 meses):

- a. **Modelo Challenger:** Entrenar versiones alternativas y compararlas automáticamente.
- b. **A/B Testing:** Probar mejoras con subconjuntos de usuarios.
- c. **MLOps Completo (CI/CD/CT):** Automatizar el pipeline end-to-end con GitHub Actions para lograr Integración, Despliegue y Entrenamiento Continuos.

6. Referencias

- Repositorio del Equipo 53 en GitHub:
<https://github.com/JCGarcesDC/ObesityMine53.git> (Rama dev3)
- Servidor de Tracking de MLflow: <https://docs.databricks.com/aws/en/workspace/>
- Dataset Original: Palechor, F. M., & de la Hoz Manotas, A. (2019). Estimation of Obesity Levels Based on Eating Habits and Physical Condition. UCI Machine Learning Repository.
- Beck, K. (2002). *Test-driven development: By example*. Addison-Wesley Professional.
- Fowler, M. (2020). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.

- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). Wiley.
- Osherove, R. (2015). *The art of unit testing: With examples in .NET* (2nd ed.). Manning Publications.
- Breck, E., Polyzotis, N., Whang, S., Roy, S., & Zinkevich, M. (2020). *The ML test score: A rubric for ML production readiness*. Google Research.
- Lakshmanan, V., Tigani, J., & Warden, P. (2020). *Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps*. O'Reilly Media.
- Lu, N. (2023). *Data drift in machine learning systems*. Springer.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). *Hidden technical debt in machine learning systems*. In *Advances in Neural Information Processing Systems*.